

# Lecture 5: Binary Search Trees

## Introduction

A Binary Search Tree (BST) is a node-based binary tree data structure where each node has a comparable key, and satisfies the property that the key in each node is greater than all keys in its left subtree and less than all keys in its right subtree.

## Insertion

To insert a new key into a BST, we start at the root and compare the new key with the current node's key. If the new key is smaller, we move to the left child; if larger, we move to the right child. We repeat this process until we find an empty spot, where the new node is attached.

## Searching

Searching in a BST follows the same comparison logic as insertion. Starting at the root, we compare the target value with the current node. If it matches, we are done. If the target is smaller, we search the left subtree; if larger, we search the right subtree. This gives an average time complexity of  $O(\log n)$  for a balanced tree.

## Time Complexity Summary

For a balanced BST, insertion, deletion, and search all run in  $O(\log n)$  time. However, in the worst case (a degenerate tree resembling a linked list), these operations degrade to  $O(n)$ . This is why self-balancing variants such as AVL trees and Red-Black trees were developed.

## Traversal Methods

Common traversal methods for a BST include in-order, pre-order, and post-order traversal. In-order traversal of a BST visits nodes in ascending order of their keys, which is a useful property for sorted output.